

Business Harness Bench: Evaluating Agents on Business Deliverables

ERP AI

Benchmark and evaluation infrastructure

github.com/erphq/business-bench

Abstract | Business agents are useful when the work they deliver is correct, usable, and repeatable. We introduce Business Harness Bench, a runnable benchmark comprising 187 file-based desk tasks across seven categories and 20 application-building tasks with three successive change requests. Desk tasks combine heterogeneous inputs, explicit business rules, reference solutions, and executable checks; build tasks evaluate delivery, permissions, data integrity, and behavior under change. We report a complete desk comparison of Proto with DeepSeek V4.1 Flash and Codex with GPT-5.6-sol, with three attempts per task and a shared frozen scorer. Proto passes 507 of 561 attempts (90.4%), compared with 473 of 561 (84.3%) for Codex, a difference of 6.06 percentage points. Captured-usage cost estimates are \$22.06 and \$219.13, respectively. These are results for two configured agent systems on a development task set, not isolated model or harness effects. The release includes task packs, runners, the frozen scorer, and an attempt-level evidence ledger; application tasks are released without a completed build leaderboard.

Full desk evaluation: 187 tasks × 3 attempts per system

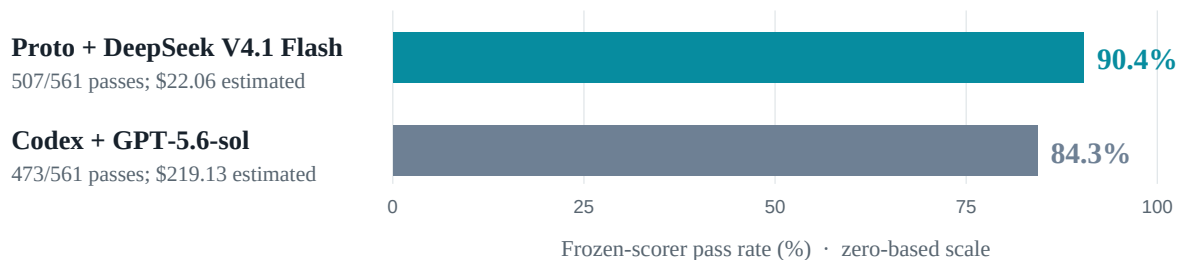


Figure 1 | Complete desk comparison under one frozen scorer. All 1,122 attempts are retained. Cost annotations are captured-usage API-equivalent estimates, not billed charges.

1. Introduction

A business owner delegates a reconciliation because a payment decision depends on it, not because a description of reconciliation is needed. The same distinction applies to an import file, a monthly report, or an internal application: the agent's response is not the deliverable. The relevant question is whether the owner can use what the agent hands back without repairing the work.

Business correctness is often conjunctive. A workbook may show the right total while its formulas fail. An import may be syntactically valid while dropping customers. A payment may match an amount but refer to the wrong invoice. In an application, hiding a button does not enforce authorization. These failures motivate evaluation against explicit delivery contracts rather than fluency, self-reported completion, or activity counts.

2. Benchmark design

Business Harness Bench tests two forms of delegation. In the **desk track**, an agent receives a folder and a short request, then produces files. In the **build track**, an agent receives a business requirement and seed data, delivers an application, and applies three changes. Both tracks evaluate the handoff; their scores remain separate because file checks and application acceptance are not interchangeable units.

2.1 Desk tasks

The desk track contains 187 tasks. Inputs include CSV and XLSX exports, text and scanned documents, contextual messages, and occasional databases. Tasks require agents to reconcile inconsistent records, apply supplied policies, preserve required fields, and distinguish supported conclusions from missing evidence. Fixtures are generated business-shaped data, not private customer engagements or a statistically representative sample of business demand.

Category	Tasks	Representative output
Spreadsheet	41	Reconciled or calculated workbook
Bookkeeping	36	Reconciliation, schedule, classification
Reports	30	Data summary and supported memo
Reformatting	26	Destination-compatible import file
Extraction	26	Structured records from documents
Drafting	19	Business text preserving supplied facts
Tooling	9	Small file-based tool or static page
Total	187	Three attempts per reported system

Table 1 | Composition of the desk track. Each task supplies an ask, workspace, generator, checks, and reference solution. Checks and references remain outside the evaluated agent's container.

2.2 Application tasks

The 20 build tasks cover CRM, inventory, orders, field service, leave, purchase approvals, time tracking, memberships, events, assets, helpdesk, vendors, expenses, quotes, donors, appointments, property maintenance, recruiting, work orders, and point of sale. A shared enterprise baseline addresses delivery, invitations, roles, server-side scope, live dashboards, exact lists and exports, audit metadata, validation, and persistence. Domain-specific checks add the business rules.

Each application receives three successive change requests. The runner carries the prior workspace forward, but invokes a new one-shot turn rather than claiming conversational continuity. Testers recheck existing requirements after each change. Automated probes support delivery checks and produce a tester sheet; they do not establish working permissions or complete acceptance. No completed build leaderboard is reported here.

2.3 Contribution and scope

The contribution is an inspectable evaluation package joining portable artifact contracts with application handoff under change. It complements enterprise interaction settings such as WorkArena [1], spreadsheet manipulation in SpreadsheetBench [2], and simulated professional work in TheAgentCompany [3]. EnterpriseClawBench [4] evaluates workspace tasks recovered from real workplace sessions. Our inputs are authored fixtures; we do not claim the same source provenance. The benchmark measures configured systems, including models, tools, skills, and execution substrate.

| 3. Evaluation protocol

3.1 Workload and configurations

The reported comparison covers all 187 desk tasks three times for each system: **561 attempts per system and 1,122 attempts in total**. Every repetition is retained. The Proto cohort uses runtime revision c8f62dd60 with DeepSeek V4.1 Flash, high reasoning, temperature 0, and DeepSeek-only routing without fallback. Codex uses CLI 0.154.0, GPT-5.6-sol, and high reasoning. Codex sampling is CLI-managed and is not asserted to match Proto's temperature.

The Proto cohort comprises one full repetition and two additional full repetitions of the same runtime. Logical repetition identifiers are preserved in the released ledger. Codex's three full repetitions are reused from its completed benchmark cohort. These are matched task sets under a shared scorer, not simultaneous randomized trials or a same-model harness ablation. Later Proto-versus-Proto development comparisons are outside this release.

3.2 Frozen artifact scoring

All attempts are scored with the same **conservative-v7** package. Each receipt binds the verdict to an original result hash, output-artifact hashes, and the scorer manifest. The release includes the immutable scorer source and task definitions. Export verifies each original result hash and receipt identity; local verification checks every file named by the frozen manifest.

Required checks are conjunctive: a task passes only when every required check passes. Workbook scoring uses native recalculation after cached values are stripped. The frozen scorer includes conservative equivalence checks for alternative valid representations. Original raw verdicts remain separately available; they are not mixed with frozen verdicts in the primary score. There are four original grader-error attempts for Proto and three for Codex, and zero errors under the frozen scorer.

3.3 Results

Measure	Proto + DeepSeek V4.1 Flash	Codex + GPT-5.6-sol
Repetition 1	171/187	152/187
Repetition 2	167/187	160/187
Repetition 3	169/187	161/187
Frozen score	507/561 (90.4%)	473/561 (84.3%)
All three attempts pass	147/187	141/187
Original raw score	447/561 (79.7%)	431/561 (76.8%)

Table 2 | Complete desk comparison. The primary score is the frozen artifact verdict. Every passing artifact also completed normally; no passing timeout inflates the normal-completion count. Proto has three timeouts overall; Codex has none.

Proto leads by 34 passing attempts, or **6.06 percentage points**. The recorded task-clustered bootstrap, which keeps each task's three repetitions together, gives a descriptive 95% interval of +1.25 to +11.05 points (20,000 resamples; seed 20260916). Related task families and development exposure limit interpretation beyond this workload.

The aggregate exceeds 90%, but that is not the same as 90% in every repetition. Proto scores 171, 167, and 169 of 187; its second repetition is 89.3%. The results support an observed lead on this full desk comparison, not a claim of universal superiority or a passed per-repetition reliability threshold.

4. Resource use

Measure	Proto + DeepSeek	Codex + Sol
Input tokens	823,587,929	165,959,457
Cached input	775,672,448	150,329,088
Uncached input	47,915,481	15,630,369
Output tokens	20,910,907	4,823,643
Estimated model cost (USD)	22.06	219.13
Median task duration (s)	124.9	135.2
p90 task duration (s)	455.6	403.3
Summed task duration (s)	113,460.1	111,218.8

Table 3 | Resources across all 561 attempts per system. Costs are captured-usage API-equivalent estimates under the recorded price table, not subscription invoices or reconciled provider bills. Summed task duration is not elapsed campaign time when attempts run concurrently.

Proto has lower estimated model cost and a lower median task duration, but performs substantially more token work. Its total task duration is slightly larger and its p90 is worse. The cost result therefore should not be restated as uniformly lower resource consumption. Cached fractions are token-weighted; the associated uncached totals are reported rather than inferred from hit rates alone.

Proto records 16,954 completed model-response records. Codex usage is captured as 561 per-task aggregates, not 561 API requests. Separate reasoning-token counts are unavailable for Codex, so a cross-system request-count or reasoning-token comparison is not justified. Nine Proto request starts lack completed responses; their additional usage is unknown, not established as zero. Transport first-token latency and turn-start versus intra-turn cache splits are not available in this report.

5. Validity and limitations

Task validity. Most task authoring used model assistance. Mechanical checks provide positive and negative controls but do not replace independent practitioner review or a human performance baseline. The task set was developed by an organization that also develops Proto, and benchmark-driven development limits claims about unseen generalization.

Scoring coverage. A shared deterministic scorer removes one source of comparison drift, not every source of measurement error. The frozen checks are not exhaustive manual acceptance of every artifact. Supplemental visual or semantic findings remain separate from the primary score. No build-track result is inferred from desk performance.

Configuration and timing. The systems use different models and different sampling controls, and the cohorts were not run contemporaneously. Provider behavior, tools, skills, and runtime choices may all contribute. The result is an end-to-end configuration comparison; it does not identify an isolated causal harness effect.

Reproduction. Source, task definitions, hashes, and score records are released. Private credentials, session traces, historical agent binaries, and raw generated artifacts are not. Artifact hashes identify the retained evidence but cannot reconstruct it. The scorer can evaluate independently supplied workspaces, and the runners support new experiments with explicitly recorded configurations.

| 6. Conclusion

Business Harness Bench evaluates whether agents satisfy concrete business-delivery contracts. In the complete reported desk comparison, Proto with DeepSeek V4.1 Flash achieves 90.4% versus 84.3% for Codex with GPT-5.6-sol under a shared frozen scorer. The release makes this result inspectable while distinguishing artifact correctness, repeated success, execution status, and resource cost. Its application track extends the same handoff principle to working systems and subsequent changes, with acceptance results left unclaimed until testing is complete.

| Appendix A. Running and auditing the benchmark

A.1 Installation and execution

Use Python 3.11 or 3.12 and Docker on a dedicated Linux evaluation host. Install requirements.txt and build the agent and recalculation images with docker/build.sh. Provision a benchmark-owned model login; private agent runtimes are supplied separately. The README contains full commands for each supported adapter and both tracks.

The desk runner mounts only the workspace, isolated home, and read-only adapter into the agent container. References remain on the host. Local-process mode is for development: it does not isolate host-side answers. Network access remains enabled for model requests and permitted tooling; containers are not a guarantee against online discovery of an exposed task set.

Use a fresh result label for each run. The runner refuses to overwrite existing desk labels or attempt directories. Build runs require an explicitly allocated port and a dedicated test environment. Do not expose unreviewed generated applications or mount production credentials. Build tester sheets include generated logins and must remain private.

A.2 Scoring and verification

```
python -m unittest discover -s tests -v
python bench/export_campaign.py --verify
python bench/release_manifest.py
python scoring/frozen-v7/scorer.py TASK_ID WORKSPACE
```

Set BENCH_RECALC_DOCKER_IMAGE to the native recalculation image when scoring workbooks. The last command uses the frozen scorer directly; bench/run.py records the original grader's result. Preserve both fields when publishing a new frozen-scored campaign. Never silently substitute a subset of rescored attempts into a raw aggregate.

The canonical manifest digest is:

b4720db00f2a55461ca70767a4baf8dd2d25aa9d301e8fe22ea13d21ccca358f

The package preserves its original assembly-status text to keep its fingerprint immutable; the subsequent recorded replay covered 1,341 attempts with zero verdict differences. Current export verification checks the receipt and source hashes rather than editing that historical manifest.

A.3 Release validation

All 187 desk reference-solution/untouched-workspace checks passed with native recalculation; all 20 build-task packs passed structural and seed validation. The strict desk validator reported 28 supplied-fixture byte differences from regenerated outputs despite deterministic repeated regeneration. The supplied inputs remain authoritative and unchanged. Affected tasks are listed in docs/validation.md; byte differences are not assumed semantically harmless.

| References

1. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? arXiv:2403.07718, 2024. <https://arxiv.org/abs/2403.07718>
2. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. arXiv:2406.14991, 2024. <https://arxiv.org/abs/2406.14991>
3. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. arXiv:2412.14161, 2024. <https://arxiv.org/abs/2412.14161>
4. EnterpriseClawBench: Benchmarking Agents from Real Workplace Sessions. arXiv:2606.23654, 2026. <https://arxiv.org/abs/2606.23654>